

Project Proposal: Full-Stack To-Do List Web Application

Python Programming — Course Project

Introduction

For my Python Programming project I will build and deploy a full-stack to-do list website using the Django web framework. The application will let a user register an account, log in, and create, view, update, and delete their own tasks, with each user only able to see their own list. I am basing the project on a nine-part video series (“Django Fullstack Todolist Website” by THE PROTON GUY) that walks through the build from setup to deployment. A to-do list is a great first project because the core idea is simple and familiar, but completing it still requires me to practice the fundamentals of Python and apply them to a real, working program rather than isolated exercises. By the end I will have a live website I can share, along with hands-on experience in how a real web application is structured. Beyond the tutorial’s scope, I also extended the finished app with an AI-powered quick-add feature, a custom retrowave-themed interface, Calendar and Lists pages for reviewing completed and categorized tasks, and a points-and-rewards system for completing tasks, described in Section 5.

Plan of Action

I will follow the playlist in order, treating each video as a milestone and testing my code after every step before moving on:

1. Set up the project: install Python and Django, create a virtual environment, start the Django project and app, and run the development server to confirm it works.
2. Build the templates and views: create the base HTML pages and connect them to Django views and URLs so the site renders in the browser.
3. Add user authentication: build register, login, and logout functionality so each person has their own account.
4. Create tasks: design the Task model and a form so a logged-in user can add new to-do items to the database.
5. Display tasks: query the database and show each user’s task list on the page.
6. Update and delete tasks: add the ability to mark a task complete and remove tasks from the list.
7. Restrict access: make sure users can only see and edit their own tasks, not anyone else’s.
8. Deploy the website: push the finished project online so it is accessible as a live link, then do final testing and write up my documentation.

Technologies Used

- **Python:** The core programming language for all of the application logic.
- **Django:** A Python web framework that handles URLs, views, forms, the database, and user authentication.
- **SQLite:** The lightweight database (built into Django) that stores user accounts and tasks.
- **HTML & CSS:** Used to build and style the web pages the user sees and interacts with.

- **Git & GitHub:** For version control and backing up my code as I follow each step.
- **Hosting platform:** A deployment service to publish the finished site online as a live, shareable link.
- **OpenAI API (ChatGPT):** One of three AI providers powering the AI quick-add feature, parsing free-text task descriptions into a structured title, description, due date, and category.
- **Google Gemini API:** A second cloud AI provider for quick-add, offered as an alternative to OpenAI; each user picks whichever provider (including the local model below) they prefer.
- **python-dotenv:** Loads API keys, email credentials, and other secrets from a local, gitignored `.env` file instead of hardcoding them in the source.
- **Bootswatch (via jsDelivr CDN):** Supplies the real, hosted CSS for every purchasable look in the Theme Shop, loaded client-side rather than authored or hardcoded into the app.
- **Django's email framework:** Sends password-reset emails — printed to the console in development (no setup required), or delivered for real over SMTP (e.g. Gmail with an App Password) once configured.
- **django-allauth:** Adds an optional “Sign in with Google” OAuth flow alongside the app's own username/password authentication.
- **llama-cpp-python:** Runs a small open-weight language model directly on the user's machine as a third, key-free quick-add provider, downloading and caching the model from Hugging Face on first use.

Timeline

As a summer project, I plan to complete the build in 10 days, working on it intensively each day:

Day	Focus	Deliverable
Day 1	Install Python and Django, set up the project and virtual environment	Running Django development server
Day 2	Build templates, views, and URLs	Base pages render in the browser
Day 3	Add user authentication — register and login	Users can create an account and log in
Day 4	Finish authentication — logout and session handling	Complete account system
Day 5	Create the Task model and add-task form	Logged-in users can add tasks
Day 6	Display each user's task list from the database	Tasks show on the page
Day 7	Update and delete tasks (mark complete, remove)	Full task management features
Day 8	Restrict tasks so users only see their own; styling	Secure, styled application
Day 9	Testing and bug fixes	Polished, tested application
Day 10	Deploy online and write documentation	Live website + final write-up

Enhancements Beyond the Original Plan

After completing the core tutorial-based build, I added nine features that go beyond the original video series:

- **AI-powered quick-add:** A natural-language entry box on the task list lets me type a plain-English note (e.g. “remind me to call mom tomorrow at 5pm”). The note is sent server-side to the OpenAI API, the Google Gemini API, or a model running entirely on the user’s own machine, depending on which provider the user has selected, which parses it into a structured task — title, optional description, optional due date, and a category — using a single, low-cost (or, for the local model, free) call. I added `due_date` and `category` fields to the `Task` model to support this. If no API key is configured, or the call fails for any reason, the feature falls back gracefully and still creates a task from the raw text, so the app never breaks.
- **Per-user API keys and provider choice:** Rather than relying on one shared key, each user can add, change, or remove their own OpenAI and/or Gemini API key from a dedicated Settings page (or optionally during registration), and pick which provider powers their quick-add via a “Preferred AI provider” selector. Keys are stored per user and only ever shown back masked, with just the last four characters visible (e.g. “ending in 3456”); a per-user key takes priority over a shared key set in `.env`. A third selector option, **Local model**, runs quick-add parsing on-device via `llama-cpp-python` instead of calling either cloud API — it needs no key at all, since the model file (downloaded once from Hugging Face and cached, or pointed at a file already on disk) runs locally on the user’s own hardware.
- **Due-date dropdowns:** The due-date field on the task form is a set of Month, Day, Year, Hour, Minute, and AM/PM dropdown selects rather than a single raw date/time input, built with a custom Django `MultiWidget`. Leaving all six blank means no due date; filling in only some of the six is rejected with a validation error instead of silently saving a partial date.
- **Retrowave visual theme:** I restyled the entire site with a synthwave/retrowave aesthetic — a dark purple-to-navy gradient background with a faint neon grid, glowing pink and cyan headings and buttons, and the Audiowide and Rajdhani Google Fonts — in place of the plain styling shown in the tutorial.
- **Calendar page:** A dedicated page lists every completed task grouped by the day it was finished, showing the exact time of completion. This is backed by a new `completed_at` timestamp on the `Task` model that is set and cleared automatically whenever a task’s complete status is toggled.
- **Lists page:** A second page groups all of a user’s tasks by category (Work, Personal, Shopping, Health, Errands, Other, or Uncategorized). Categories are assigned automatically by the AI quick-add feature, or can be set by hand from a task’s edit form.
- **Points and Theme Shop:** Completing a task for the first time now earns 10 points, shown in the navbar; toggling a task off and back on again does not pay out a second time, tracked via a new `points_awarded` flag on `Task`. Points can be spent on a new Theme Shop page to unlock additional site-wide looks beyond the default retrowave theme. Rather than hand-authoring each paid theme’s CSS, every option’s actual stylesheet is fetched live, in the visitor’s browser, from Bootswatch (bootswatch.com) over its jsDelivr CDN — the app itself only stores which theme key maps to which CDN URL and its point cost.
- **Password reset via email:** A “Forgot your password?” link on the login page lets a user request a one-time reset link sent to their account email, using Django’s built-in password-reset views and a custom set of templates styled to match the retrowave theme. In development the email prints to the console; in production it can be sent for real over SMTP (e.g. Gmail with an App Password) by setting a few values in `.env`, with no code changes required. Requesting a reset for an email that isn’t registered returns the same confirmation page as a real one, so the flow can’t be used to probe which addresses have accounts.

- **Google sign-in:** An optional “Sign in with Google” button on the login and register pages, built on `django-allauth`, lets a user authenticate with a Google account instead of a password. The button only appears once a Google OAuth Client ID/Secret are configured in `.env`; without them, password-based registration and login work exactly as before. The first time a given Google account signs in, a local account and profile are created automatically, with no separate username-picking step.

All nine additions are covered by automated Django tests; all three quick-add providers' calls are mocked (including the local model's loading and inference), password-reset emails are captured in Django's test outbox rather than actually sent, and the Google sign-in tests never contact Google — so running the suite never makes a real network request, sends real email, downloads a model, or incurs API cost.